

Applesoft's Loaded Dice

The Truth About Randomness on the Apple II, Then and Now

Jeff Robison · VCF Midwest 21 2026

Expanded edition · draft for Jeff Robison's review

The Problem

1

```
] PRINT RND(1)  
.973136996
```

Power off. Power on. Same number.

“...exactly the same sequence each time the machine is powered on.”

Why nobody noticed

Everything a programmer does at a desk looks random.

RUN it again

Different numbers.

The seed carries on from the last call.

Reboot DOS, Ctrl-Reset

Different numbers.

The seed survives in RAM.

Power switch

The same sequence.

The first user of the day sees it.

Pseudo-random means repeatable

The seed

Where the sequence starts.

Needs entropy: something you can't predict.

The generator

What the sequence looks like.

Needs a long period and no visible pattern.

Same seed → same numbers. Every time. This talk is about the seed.

Integer BASIC, 1977: seeded by you

Woz's generator at \$EF4E, hand-assembled.

The generator

15-bit shift register

Repeats every 32,767 calls

$RND(X) = \text{state} \bmod X$

50 bytes

The seed

\$4E/\$4F counts while the monitor waits for a key

That counter is the generator's state

Every key wait stirs it

6 bytes, in KEYIN

Good enough for games, its stated purpose. Not for statistics.

The entropy source is a busy-wait

Not a timer. Not a jiffy clock. A loop count.

```

fd1b: e6 4e      KEYIN      inc    RNDL
fd1d: d0 02            bne    KEYIN2      ;incr rnd number
fd1f: e6 4f            inc    RNDH
fd21: 2c 00 c0      KEYIN2    bit    IOADR      ;key down?
fd24: 10 f5            bpl    KEYIN        ; loop
fd26: 91 28            sta    (BASL),y    ;replace flashing screen
fd28: ad 00 c0            lda    IOADR      ;get keycode
fd2b: 2c 10 c0            bit    KBDSTRB    ;clr key strobe
fd2e: 60            rts

```

Monitor KEYIN, \$FD1B. Listing credited to S. Wozniak and A. Baum.

Rate

15 CPU cycles per pass
 ≈ 68,000 counts a second
 Wraps ≈ once a second

Same counter in

Autostart F8 (\$FD1B)
 Ile 80-col (\$CB15)
 Ilc (\$CC71)

Only moves

while a KEYIN-style routine waits for a key

What the manual promised

RND (n)

If \aexpr\ is greater than zero, RND(aexpr) generates a new random number each time it is used.

RND (-n)

If \aexpr\ is less than zero, RND(aexpr) generates the same random number each time it is used with the same \aexpr\, as if from a permanent random number table built into the APPLE. If a particular negative argument is used to generate a random number, then subsequent random numbers generated with positive arguments will follow the same sequence each time. A different random sequence is initialized by each different negative argument. The primary reason for using a negative argument for RND is to initialize (or "seed") a repeatable sequence of random numbers. This is particularly helpful in debugging programs that use RND.

RND (0)

If \aexpr\ is zero, RND(aexpr) returns the most recent previous random number generated (CLEAR and NEW do not affect this). Sometimes this is easier than assigning the last random number to a variable in order to save it.

.973136996

...starting here, every power-on.

**Repeatable on purpose:
documented.**

**Repeatable by accident:
not.**

Applesoft II: Microsoft's RND

Microsoft wrote it. Apple shipped it unchanged through the enhanced IIe.

```

*****
* RND statement
*****
efae: 20 82 eb  RND      jsr     SIGN      ;reduce argument to -1, 0, or +1
efb1: aa        tax      ;save argument
efb2: 30 18      bmi     LEFCC          ;= -1, use current argument for seed
efb4: a9 c9      lda     #RNDSEED      ;use current seed
efb6: a0 00      ldy     #>RNDSEED
efb8: 20 f9 ea  jsr     LOAD_FAC_FROM_YA
efbb: 8a        txa      ;recall sign of argument
efbc: f0 e7      beq     RTS_19         ;=0, return seed unchanged
efbe: a9 a6      lda     #<CON_RND_1    ;very poor RND algorithm
efc0: a0 ef      ldy     #>CON_RND_1
efc2: 20 7f e9  jsr     FMULT
efc5: a9 aa      lda     #<CON_RND_2    ;also, constants are truncated
efc7: a0 ef      ldy     #>CON_RND_2    ;<<< this does nothing, due to small exponent >>>
efc9: 20 be e7  jsr     FADD
efcc: a6 a1      LEFCC   ldx     FAC+4    ;shuffle hi and lo bytes
efce: a5 9e      lda     FAC+1          ;to supposedly make it more random
efd0: 85 a1      sta     FAC+4
efd2: 86 9e      stx     FAC+1
efd4: a9 00      lda     #$00          ;make it positive
efd6: 85 a2      sta     FAC_SIGN
efd8: a5 9d      lda     FAC           ;a somewhat random extension
efda: 85 ac      sta     FAC_EXTENSION
efdc: a9 80      lda     #$80          ;exponent to make value < 1.0
efde: 85 9d      sta     FAC
efe0: 20 2e e8  jsr     NORMALIZE_FAC_2
efe3: a2 c9      ldx     #RNDSEED      ;move FAC to RNDSEED
efe5: a0 00      ldy     #>RNDSEED
efe7: 4c 2b eb  GO_MVMF  jmp     STORE_FAC_AT_YX_ROUNDED

```

Seed at \$C9

Filled from ROM at power-on.
Nothing outside RND touches it.

\$4E/\$4F

Still counts.
Applesoft never reads it.

II Plus ROM. Comments: Bob Sander-Cederlof (S-C DocuMentor), via Andy McFadden. Not Microsoft's.

One byte short

Cold start copies 4 of the 5 seed bytes. The fifth, \$CD, is whatever RAM held.

```
F123: 80 4F C7 52 58    seed table

F150: A2 1C            LDX #$1C        ; one short
F152: BD 0A F1        LDA $F10A,X
F155: 95 B0            STA $B0,X
F157: 86 F1            STX SPEEDZ      ; Apple's
F159: CA              DEX
F15A: D0 F6            BNE $F152
```

\$CD at power-on

First RND(1)

\$FE or \$FF

.973136996

\$00

.270011996

\$58 (intended)

.512199496

\$AA

.738761996

256 possible bytes → 181 different first numbers. Each machine still repeats its own.

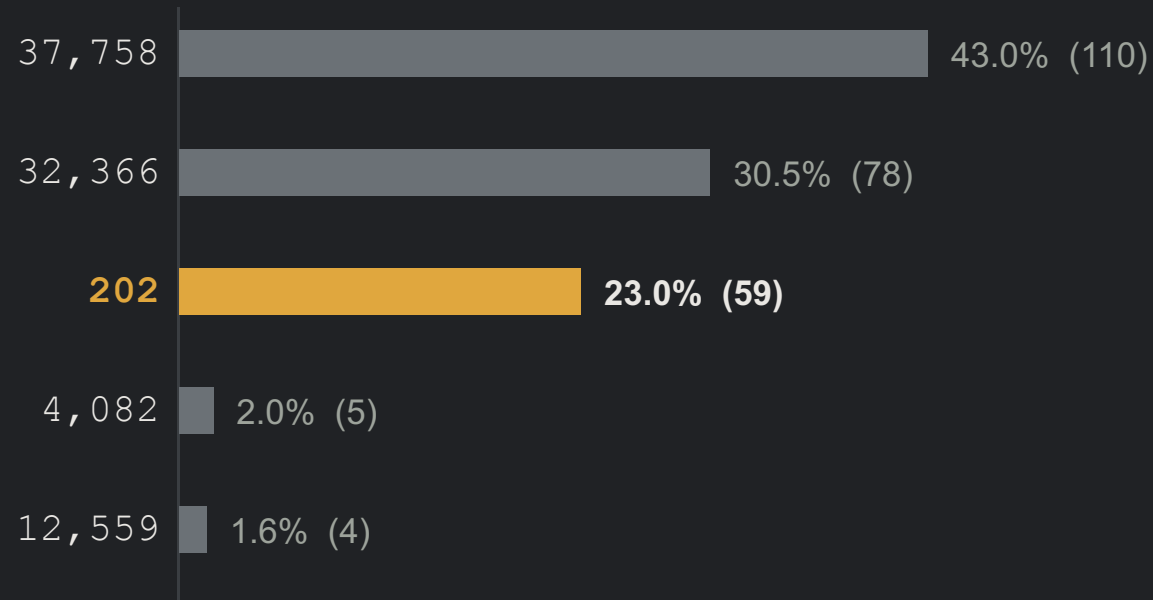
Two published loops, one generator

Kaner & Vokey saw a loop of 202. Sander-Cederlof saw 37,758. Both were right.

Where all 256 possible cold starts end up, by loop length

Share of the 256 values the uncopied byte \$CD can hold

Loop length



\$CD = \$FF

37,758-number loop
after 6,818 calls

\$CD = \$58, the intended byte

202-number loop
after 15,382 calls

Reseeding

RND(-52894) lands in the 202 loop
too

A regression of the default

Both generators are weak. Only one seeded itself.

	Integer BASIC (1977)	Applesoft II (Microsoft)
State	15 bits at \$4E/\$4F	5-byte float at \$C9–\$CD
Loop length	32,767, one cycle	202 to 37,758, depends on seed
Seed by default	Keypress timing, automatically	ROM, plus one leftover byte
Reads \$4E/\$4F	It is \$4E/\$4F	Never
Programmer must	Nothing	RND(-(PEEK(78)+256*PEEK(79)))

The machine didn't lose its entropy source in 1978. The default stopped using it.

Why 1978 got worse

- ▶ Microsoft's BASIC had two RNDs: timer-seeded for Commodore, a fixed seed for everyone else.
- ▶ Apple got the everyone-else build.
- ▶ Apple edited code inside the seed-copy loop (STX SPEEDZ at \$F157) and kept the short count.
- ▶ The same RND bytes shipped in the II Plus, IIe and enhanced IIe ROMs.

No record explains why. Nobody connected the counter the monitor was already running.

The damage

12

“...the subjects receiving the repeated lists were those tested at the beginning of each day, immediately after the computer had been turned on.”

Aldridge, 1987: a memory experiment with “individually randomized” word lists

- ▶ Kaner & Vokey needed RND to randomize experiments too.
- ▶ Published research ran on this generator.

Found, reported, worked around

13

Year	Who	What
1982/84	Kaner & Vokey, MICRO	Loop of 202 numbers; replacement generator
1983	Call-A.P.P.L.E.	“RND is Fatally Flawed,” pp. 29–34
1984	Sander-Cederlof, AAL	Seed copies 4 of 5 bytes; repetition at 37,758
1987	Aldridge, BRMIC	Same word lists every morning; Apple’s advice: seed from \$4E/\$4F
1988	Gleason, Collegiate Micro.	Statistical tests; suggested seeds
1989/99	Moore; Empson	Shift-register replacement seeded from \$4E/\$4F

Found in 1982. Worked around, one program at a time. Never fixed in ROM.

“Just seed it first”

Right. That's what Apple told Aldridge's lab in 1987.

New code

```
X = RND ( - ( PEEK ( 78 ) + 256 * PEEK ( 79 ) ) )
```

after a key wait, before the first RND

Existing code

You can't edit software you didn't write.

The failure is silent: only people who already know can fix it.

This machine used to do it automatically.

The patch is for the programs nobody will ever edit.

The fix: put the counter back

Copy the ROM into the language card, patch the copy, read from the card.

\$F800–\$FFFF	Monitor, copied
\$F601	DRAW, untouched
\$F600	RTS shared with HLIN, untouched
\$F5CB–\$F5FF	Patch code over HFIND: 53 bytes, never called by Applesoft
\$EFAE	RND entry: JMP \$F5CB
\$D000–\$F7FF	Applesoft, copied

Language card RAM, top to bottom. Not to scale.

Loader steps

1. Write-enable the LC, ROM still readable
2. Copy \$D000–\$FFFF into the LC
3. Write the patch at \$F5CB
4. Write JMP \$F5CB at \$EFAE
5. Switch the LC to read

What the fix does, and doesn't

Does

- Positive RND draws on the KEYIN counter
- RND(-n) still repeats; RND(0) still replays
- No program changes

Doesn't

- Fix the generator: the 202 loop is still there
- Help before the first key wait
- Work without a language card

[Jeff: confirm behavior and compatibility (backup B4) before this slide is shown]

```
[power off, power on]  
]BRUN LC_LOADER  
]PRINT RND(1)
```

```
[power off, power on]  
]BRUN LC_LOADER  
]PRINT RND(1)
```

Two power-ons. Two different numbers.

Live, or a recorded clip if power-cycling on stage is risky.

Now: the same shape, 2006

A change silently disconnects the entropy. The output still looks random.

Debian OpenSSL, 2006–2008

A Debian change removed nearly all entropy from OpenSSL's PRNG

Keys were predictable until May 2008

CVE-2008-0166

C, today

rand() without srand() starts as if seeded with 1

Same sequence, every run

A parallel, not a lineage.

1977: Integer BASIC seeded RND from you. Applesoft seeded it from ROM.
It hid for years: rerun looked random; only power-on repeated.
The fix puts the counter back without changing a single program.

Try it yourself

[URL: Jeff to supply]

QR
Jeff to supply

Backup: Applesoft RND, byte by byte

B1

```
EF AE: 20 82 EB JSR SIGN      ; -1 / 0 / +1
EF B1: AA      TAX
EF B2: 30 18    BMI $EFCC      ; negative: reseed from arg
EF B4: A9 C9    LDA #$C9
EF B6: A0 00    LDY #$00
EF B8: 20 F9 EA JSR LOAD_FAC   ; FAC = seed
EF BB: 8A      TXA
EF BC: F0 E7    BEQ $EFA5      ; RND(0): return seed
EF BE: A9 A6    LDA #$A6
EF C0: A0 EF    LDY #$EF
EF C2: 20 7F E9 JSR FMULT      ; x 11879546.40625
EF C5: A9 AA    LDA #$AA
EF C7: A0 EF    LDY #$EF
EF C9: 20 BE E7 JSR FADD       ; + 3.93E-8, lost
EF CC: A6 A1    LDX FAC+4      ; swap highest and
EF CE: A5 9E    LDA FAC+1      ; lowest bytes
EF D0: 85 A1    STA FAC+4
EF D2: 86 9E    STX FAC+1
EF D4: A9 00    LDA #$00
EF D6: 85 A2    STA FAC_SIGN   ; force positive
EF D8: A5 9D    LDA FAC
EF DA: 85 AC    STA FAC_EXT    ; old exponent -> guard
EF DC: A9 80    LDA #$80
EF DE: 85 9D    STA FAC       ; value < 1
EF E0: 20 2E E8 JSR NORMALIZE
EF E3: A2 C9    LDX #$C9
EF E5: A0 00    LDY #$00
EF E7: 4C 2B EB JMP STORE_FAC ; round, write $C9-$CD
```

Not a textbook LCG

No modulus.

Float multiply, byte swap, renormalize.

LCG lattice theory doesn't transfer.

Measured

Fraction bit 25 is set in 99.75% of outputs.

Leading digits pass uniformity and serial-pair χ^2 .

Backup: the constants, as actually read

B2

Constant	Stored at	Bytes actually read	Value used
Multiplier	\$EFA6	98 35 44 7A 68	11,879,546.40625
Addend	\$EFAA	68 28 B1 46 20	3.93×10^{-8}
Seed	\$F123	80 4F C7 52 (58 never copied)	$\approx .811635157$ + leftover byte

The addend changed the stored seed in 5 of 57,021 steps. “Effectively nothing,” not “nothing.”

Backup: \$CD on real hardware (to fill in)

B3

Machine / ROM	Boot path	PEEK(205)	First RND(1)
II Plus	No disk, Ctrl-Reset to]		
II Plus	DOS 3.3		
IIe (enhanced)	DOS 3.3		
IIe (enhanced)	ProDOS / BASIC.SYSTEM		
IIc	ProDOS / BASIC.SYSTEM		

Emulator reference: \$FF → .973136996 · \$00 → .270011996 · \$58 → .512199496

Backup: patch compatibility (to fill in)

B4

Configuration	Risk	Result
ProDOS 8 / BASIC.SYSTEM	ProDOS lives in LC RAM; loader overwrites it	untested
DOS 3.3, FP / INT	LC switches flip; patch may silently vanish	untested
Original II, Integer in ROM	Copies Integer BASIC, JMPs into its middle	untested
IIe / IIc, Ctrl-Reset	RESET forces ROM read; patch off	untested
Programs using the LC	Pascal, CP/M, RAM disks, /RAM	untested
//c ROMs, IIc Plus, IIgs	Applesoft bytes not checked	untested

Backup: HFINd and \$F5CB

- ▶ HFINd runs \$F5CB–\$F5FF: 53 bytes. “Not called by any Applesoft routine” (S-C DocuMentor).
- ▶ No JSR or JMP to \$F5CB anywhere in the II Plus ROM (byte search).
- ▶ \$F600 is an RTS that HFINd reaches from BEQ at \$F59C. Overwrite it and HPLOT TO breaks.
- ▶ DRAW starts at \$F601.
- ▶ Residual risk: a program that does CALL 62923 itself.
- ▶ Regression test after patching: HPLOT ... TO, DRAW, XDRAW, SCALE, ROT.

Backup: Commodore, same generator, different integration

B6

Same

Microsoft's generator and constants

Seed-copy bug in every Microsoft 6502 BASIC

Different

Microsoft's Commodore build: RND(0) reads hardware timers (PET, 1977)

A TI clock variable

Idiom: ``RND(-TI)``

Commodore got the platform integration from Microsoft. Apple did its own, and skipped RND.

Backup: sources

B7

- ▶ Microsoft, BASIC M6502 8K v1.1 source, github.com/microsoft/BASIC-M6502 (2025)
- ▶ Sander-Cederlof, S-C DocuMentor: Applesoft; McFadden, 6502disassembly.com
- ▶ Kaner & Vokey, “A Better Random Number Generator for Apple’s Floating Point BASIC,” MICRO, June 1984
- ▶ Call-A.P.P.L.E., “RND is Fatally Flawed,” Jan 1983, pp. 29–34
- ▶ Sander-Cederlof, “Random Numbers for Applesoft,” Apple Assembly Line, May 1984
- ▶ Aldridge, “Cautions regarding random number generation on the Apple II,” BRMIC 19(4), 1987
- ▶ Gleason, Collegiate Microcomputer 6(2), 1988 (abstract read)
- ▶ Empson, GS WorldView, Nov 1999 (on Moore, 1989)
- ▶ Apple II Reference Manual (1978); Applesoft II BASIC Programming Reference Manual (1978)
- ▶ Steil, pagetable.com and [msbasic](https://msbasic.com); Weyhrich, Apple II History